

Temporal Memory-Aware Online Test-Time Adaptation on Dynamic Graphs

Bo Li¹, Xin Zheng², Ming Jin¹, Can Wang¹, Shirui Pan^{1,*}

¹ Griffith University, Gold Coast, Australia

² RMIT University, Melbourne, Australia

{bo.li, ming.jin, can.wang, s.pan}@griffith.edu.au, xin.zheng2@rmit.edu.au

Abstract—Test-time adaptation (TTA) on graphs aims to adapt a graph neural network (GNN) that is well-trained on the training graph to the test graph, which involves potential distribution shifts that may harm model generalization and test-time inference. While recent efforts have investigated TTA on static graphs, there is still a research gap on dynamic graphs learned with dynamic GNN (DGNN) models, where both structural connectivity and node semantics evolve continuously over time. This makes adapting a DGNN model for reliable test-time performance substantially challenging. To fill this gap, in this work, we propose a novel framework of temporal memory-aware Online Test-Time Adaptation on Dynamic Graphs, named **DGOTTA**, to effectively adapt well-trained DGNNs during test time. Specifically, the proposed DGOTTA contains three modules: (1) *temporal-aware augmentation*, to extend the diversity of test dynamic graphs for addressing complex temporal and spatial shifts; (2) *memory-aware model prediction*, to alleviate catastrophic forgetting; (3) *consistency-guided online adaptation*, to enforce temporal alignment and memory smoothness. Extensive experiments on three real-world datasets and four DGNN backbones demonstrate that **DGOTTA** significantly improves generalization under diverse distribution shifts and multiple model architectures.

I. INTRODUCTION

Dynamic graphs are prevalent in real-world applications such as social networks [1]–[4], financial transaction networks [5], [6], and traffic systems [7]–[10], where both the topology and node attributes evolve over time. Dynamic graph neural networks (DGNN), have been recognized as powerful models for capturing the complex interplay of spatio-temporal dependencies, and learning expressive dynamic node and interaction representations.

In this work, we characterize distribution shift in dynamic graphs as the discrepancy between training-time and test-time graph streams along both temporal and structural dimensions. Concretely, such shifts manifest as changes in graph density, node feature statistics, and edge dynamics as the graph evolves over time, rather than as i.i.d. feature perturbations.

As illustrated in Figure 1, real-world dynamic graph benchmarks such as Wikipedia, MOOC, and Reddit exhibit clear temporal discrepancies between training and test snapshots across these statistics. The test-time graph streams demonstrate evolving patterns in interaction frequency, feature distributions, and connectivity dynamics, indicating that the underlying data distribution is non-stationary and continuously drifting. These observations highlight that distribution shift in

dynamic graphs is inherently temporal and structural, which fundamentally challenges the assumption of stationary test distributions adopted by conventional DGNN inference.

Moreover, DGNNs that are well-trained on the training dynamic graphs usually show poor performance and generalization ability to new incoming unlabeled test graphs [11], due to potential distribution shifts that might harm the performance. For instance, in recommendation systems where dynamic graphs capture user-item interactions, changes in user features (e.g., an increase income) can significantly alter user preferences and behaviors (e.g., a shift towards higher-priced items). Consequently, a DGNN model trained on earlier feature distributions (e.g., lower income) may struggle to deliver relevant recommendations at test time, as it relies on outdated interactions (such as purchases of lower-cost items) [12], [13]. Despite the effectiveness of well-trained DGNNs on the training graphs, their generalization capability often deteriorates on distribution-shifted test graphs, especially when deployed in realistic dynamic environments.

As shown in Figure 1, a well-trained DGNN with fixed parameters θ_{tr}^* is directly applied to the test phase, where test dynamic graphs are inherently evolving—node semantics shift, new edges appear, and structural patterns change continuously. As a result, the traditional DGNN inference would lead to low performance with poor generalization.

To address this test-time degradation problem, test-time adaptation (TTA) methods on graphs, which aim to adapt a well-trained GNN to unseen test distributions, become a promising solution. However, existing TTA methods on graphs mainly focus on static graphs. There is still a research blank for exploring TTA on dynamic graphs with DGNNs. Specifically, there are two core challenges for TTA with DGNNs: **Challenge-1**: at the dynamic graph level, complex temporal and spatial shifts on test graphs complicate the informative representation transfer during test time, leading to poor generalization; **Challenge-2**: at the method level, temporal evolution enforces an online learning paradigm in DGNNs, where catastrophic forgetting severely degrades DGNNs’ test-time inference performance.

To address these challenges, in this work, we propose a novel temporal memory-aware Online Test-Time Adaptation method for Dynamic Graphs, named **DGOTTA**, to effectively adapt well-trained DGNNs to evolving test-time dynamic graphs with potential distribution shifts. Specifically, the pro-

* Corresponding author.

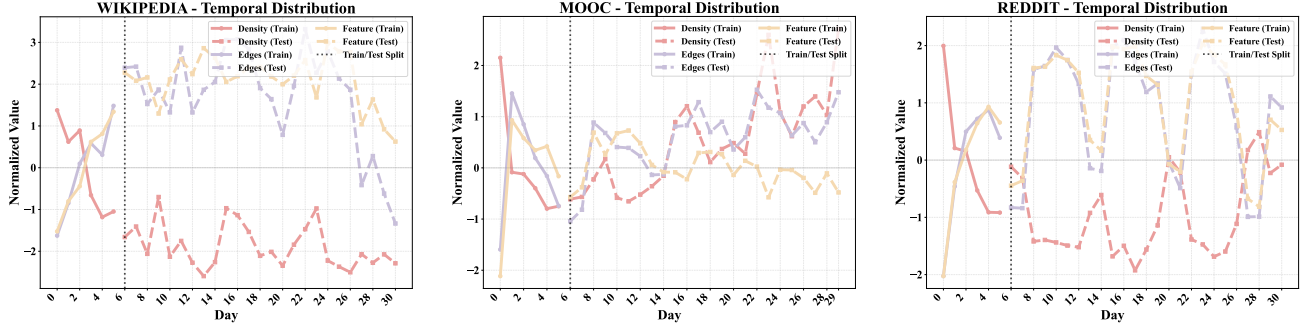
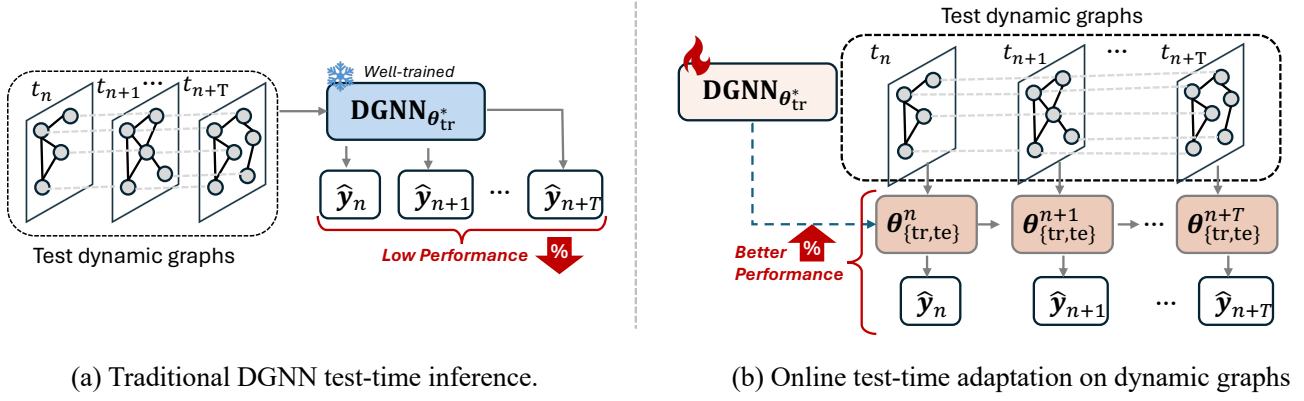


Fig. 1: Motivation and empirical evidence of temporal distribution shift in dynamic graphs. (a) Traditional DGNN test-time inference, where a well-trained model with fixed parameters is directly deployed on evolving test graph streams, leading to degraded performance under temporal and structural shifts. (b) Online test-time adaptation on dynamic graphs, which continuously updates model parameters during deployment to better align with evolving test distributions. Empirical visualization of temporal distribution discrepancies between training and test graph snapshots on Wikipedia, MOOC, and Reddit, respectively.

posed method contains three essential modules: (1) *temporal-aware augmentation*, to extend the diversity of test dynamic graphs for addressing complex temporal and spatial shifts, leading to improved robustness against temporal shifts (C1); (2) *memory-aware model prediction*, to stabilize unsupervised adaptation with the exponential moving average strategy and alleviate catastrophic forgetting (C2); and (3) *consistency-guided online adaptation*, to enforce prediction alignment across time and ensure smooth adaptation. By doing so, the proposed method could enable reliable online adaptation to dynamic graphs without access to labeled data. Extensive experiments on three real-world datasets and four DGNN backbones demonstrate that DGOTTA significantly improves generalization under diverse distribution shifts and multiple model architectures. In summary, the contributions of this work are presented as:

- To the best of our knowledge, we are the first to study online test-time adaptation for dynamic graphs, enabling models to adapt temporal and structural distribution shifts during inference in a model-agnostic manner.
- We develop a novel framework named DGOTTA, containing (1) temporal-aware augmentation; (2) memory-aware model prediction; and (3) consistency-guided online adaptation for addressing key challenges in test-time adaptation on dynamic graphs with DGNNs.

- Extensive experiments on real-world datasets and dynamic GNN backbones show that DGOTTA consistently outperforms existing baselines, achieving state-of-the-art performance under dynamic distribution shifts.

II. RELATED WORKS

Online Test-Time Adaptation. Online test-time adaptation (OTTA) aims to adapt a pretrained model from the source domain to an unlabeled target domain without re-accessing the source domain during adaptation [14], and it allows the model to adapt the test data in a source-free and online manner [15]–[17]. For image data, recent works have proposed image TTA by entropy minimization [17]–[20], pseudo-labeling [18], [20]–[22] and other like Laplacian Adjusted Maximum likelihood Estimation [23]. A subset of OTTA approaches has explored continual or online adaptation settings. Studies on this topic are focusing on the problems which target domains are not fixed but change continuously in an online manner [19], [24]–[27].

Graph Test-Time Adaptation. Studies on graph test-time adaptation (GTTA) are focusing on the problems due to the co-existence of attribute shifts and structure shifts [28]–[30]. To cope with these issues, Gtrans [31] proposes to refine the target graph at test time by minimizing a surrogate loss. SOGA [32] maximizes the mutual information between model inputs and outputs, and encourages consistency between

neighboring or structurally similar nodes, but it is only applicable to homophilic graphs [33]. In terms of degree shift, GraphPatcher [34] earns to generate virtual nodes to improve the prediction on low-degree nodes. Moreover, GAPGC [35] and GT3 [36] follow a self-supervised learning (SSL) scheme to fine-tune the pre-trained model for graph classification. There are also increasing studies on continual learning [37]. The online TTA studied in this paper differs from continual learning in that the latter updates model parameters through the fine-tuning of the model with ground-truth labels, whereas online TTA has no access to labels during test time.

Dynamic Graph Neural Networks. Dynamic graph neural networks aim to model graphs whose structures and attributes evolve over time. Existing DGNN methods usually encode temporal interactions through recurrent memory modules, temporal attention, time encoding, or message-passing mechanisms over continuous-time event streams [38]–[40]. Representative models such as TGAT [41], TGN [42], GraphMixer [43], and DyGFormer [44] have shown strong ability in learning temporal node representations and predicting future interactions or node labels. However, most DGNNs are trained under the assumption that the training and test graph streams follow similar temporal patterns. When the test stream exhibits distribution shifts, such as changes in interaction frequency, structural density, or feature statistics, a fixed DGNN may produce outdated representations and unreliable predictions [45], [46]. Different from conventional DGNN training, our work focuses on adapting a pretrained DGNN during deployment using only unlabeled test-time graph streams. This setting is more challenging because the model must update online without supervision while preserving temporal consistency.

III. PROBLEM DEFINITION

We consider the problem of online test-time adaptation for dynamic graphs. Given a dynamic graph stream $\{\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t, \mathbf{X}_t)\}_{t=1}^T$, where $\mathcal{V}_t$ denotes the set of nodes at time t , $\mathcal{E}_t \subseteq \mathcal{V}_t \times \mathcal{V}_t$ is the set of timestamped edges, and $\mathbf{X}_t \in \mathbb{R}^{|\mathcal{V}_t| \times d}$ represents the node features. The problem of online test-time adaptation on dynamic graphs aims to make reliable predictions on each unseen test graph snapshot $\mathcal{G}_t$ by adjusting well-trained DGNN models.

Formally, let f_{θ} be a well-trained DGNN model. At each test time step t , the model receives an unlabeled graph snapshot $\mathcal{G}_t$ and updates its parameters θ_t via an online adaptation strategy. The objective is to minimize the expected adaptation loss over time:

$$\min_{\theta_t} \mathbb{E}_{\mathcal{G}_t \sim \mathcal{D}_{\text{test}}} [\mathcal{L}_{\text{OTTA}}, (f_{\theta_t}(\mathcal{G}_t))] \quad (1)$$

where $\mathcal{L}_{\text{OTTA}}$ denotes the adaptation objective, such as consistency loss or temporal smoothness loss, and the model is adapted incrementally in an online fashion.

IV. METHODOLOGY

In this section, we introduce DGOTTA with temporal memory-aware online test-time adaptation on dynamic graphs.

The overall framework of our proposed DGOTTA is presented in Figure 2.

The design of DGOTTA follows two principles. First, the adaptation process should be temporally aware, since the usefulness of node features and edges depends strongly on their timestamps in dynamic graphs. This motivates temporal-aware augmentation, which produces realistic perturbed views without breaking temporal causality. Second, the adaptation process should be stable under noisy pseudo-supervision. Since no labels are available at test time, directly optimizing the model using its current predictions may amplify errors and cause confirmation bias. To address this issue, DGOTTA uses memory-aware pseudo labels and drift-aware parameter smoothing to balance short-term adaptability and long-term stability.

DGOTTA enables a deployed model to adapt to temporal and structural distribution shifts by incorporating the following three key modules:

- (1) **Temporal-Aware Augmentation.** To simulate distributional shifts, we introduce temporal-aware perturbations on node features and time-window-based sampling on edges, modeling both semantic drift and structural evolution.
- (2) **Memory-Aware Model Prediction.** A well-trained DGNN is used to generate soft pseudo labels, while an online adaptive model continuously updates its parameters using prediction feedback. A memory bank stores recent predictions to stabilize the pseudo-label generation and smooth temporal dynamics.
- (3) **Consistency-Guided Online Adaptation:** We apply prediction consistency loss between the adaptive model and pseudo labels, and a temporal consistency loss across recent predictions, jointly guiding robust online updates under evolving graph distributions.

Together, these components enable DGOTTA to continuously self-adjust to unseen graph distributions at test time, leading to better inference performance.

A. Temporal-Aware Augmentation

Graph augmentation is a crucial component of our framework, as it enhances model robustness to distribution shifts during online test-time adaptation. Unlike static graphs, temporal graphs introduce unique challenges: node interactions evolve over time and neighborhood semantics are inherently time-sensitive. Most existing augmentation strategies, however, are based on static subgraph sampling [47], implicitly assuming that all historical interactions are equally informative. This assumption neglects the central role of time in temporal graph modeling, where the recency and order of interactions fundamentally influence message passing and prediction performance, thereby motivating the need for time-aware augmentation.

For instance, a neighbor node that interacted with the target node recently may carry much more relevant information than a node that last interacted months ago. Naively removing such time-relevant edges or perturbing node features without considering their timestamps can distort local temporal dynamics and mislead the model during adaptation.

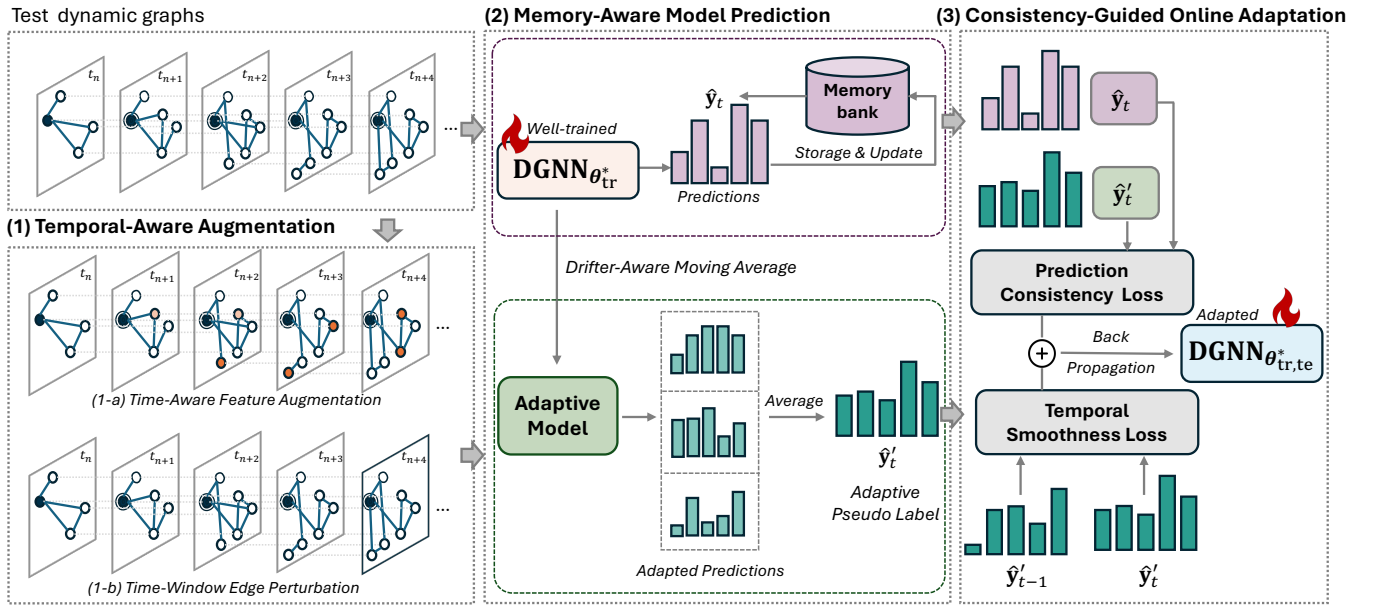


Fig. 2: **Overview of our proposed temporal memory-aware online test-time adaptation for dynamic graphs.** Given test dynamic graphs, we first conduct (1) *temporal-aware augmentation* for applying time-aware feature perturbation and time-window-based edge perturbation to simulate temporal and structural drift; Then, implement (2) *memory-aware model prediction* for generating soft pseudo labels, while the adaptive DGNN updates its parameters using a memory bank of past predictions; Last, develop (3) *consistency-guided online adaptation* to incorporate prediction consistency and temporal smoothness losses to ensure accurate online adaptation.

To address these limitations, we introduce temporal-aware augmentation, which incorporates temporal priors into both node-level and edge-level perturbations. Our augmentation simulates realistic, temporally grounded variations that help the model adapt to distribution shifts without violating the causal and temporal structure of the graph.

Time-Aware Feature Augmentation. In dynamic graphs, node features $\mathbf{x}_i$ may gradually become outdated or stale depending on the time elapsed since their last interaction. To account for this temporal degradation, we design a temporal-aware augmentation mechanism that explicitly discounts older features and injects mild noise to promote robustness.

Given a node v_i with raw feature vector $\mathbf{x}_i \in \mathbb{R}^d$ and last interaction timestamp t_i , we define the augmented feature $\tilde{\mathbf{x}}_i$ at current time T as:

$$\tilde{\mathbf{x}}_i = \mathbf{x}_i \cdot e^{-\lambda(T-t_i)} + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2), \quad (2)$$

where $\lambda > 0$ controls the temporal decay rate and σ^2 determines the level of injected Gaussian noise.

Time-Window Edge Perturbation. In temporal graphs, edge relevance naturally decays over time as interactions become outdated. To reflect this temporal locality, we introduce time-window edge perturbation, which selects and perturbs edges based on their timestamps relative to the current time. Let $\mathcal{E}_t$ denote the set of edges observed up to time t . We first extract a subset of temporally recent edges by applying a sliding time window of size W :

$$\mathcal{E}_{\text{window}} = \{e_{ij} \in \mathcal{E}_t \mid t - t_{ij} \leq W\},$$

where t_{ij} is the timestamp of edge e_{ij} . To inject structural noise and simulate uncertainty in interaction dynamics, we then perform Bernoulli sampling over the retained edges with drop probability p_{drop} :

$$\tilde{\mathcal{E}} = \mathcal{E}_{\text{window}} \mid \{e_{ij} \sim \text{Bern}(p_{\text{drop}})\}, \quad (3)$$

where each edge is independently retained with probability $1 - p_{\text{drop}}$. This augmentation mechanism prevents the model from overfitting to precise temporal topologies and encourages reliance on stable, recent structures. Compared to random edge removal in static graphs, our approach is temporally informed and causality-aware, ensuring that irrelevant or outdated interactions are not mistakenly emphasized. It is worth noting that the proposed augmentation differs from conventional random perturbation. In static graph augmentation, randomly dropping edges or masking features is often sufficient to improve robustness. However, in dynamic graphs, an edge is not only a structural connection but also a temporal event. Dropping recent edges and retaining outdated edges may distort the temporal dependency used by DGNNs for message passing. Therefore, our augmentation explicitly preserves temporal locality by first selecting recent interactions through a time window and then applying stochastic perturbation within this causally valid region. This design encourages the adaptive model to learn robust representations from plausible temporal variations rather than arbitrary graph corruption.

B. Memory-Aware Model Prediction

While temporal-aware augmentation provides diverse input views, the absence of ground-truth labels poses a fundamental

challenge to online adaptation. Without supervision, the model risks overfitting to noisy or misaligned predictions. To overcome this, we employ a dual-model prediction scheme consisting of: (a) a well-trained DGNN model that remains fixed during adaptation and serves as a stable source of pseudo-labels, and (b) an adaptive model that is continuously updated through consistency-based objectives to track the evolving graph distribution. This design enables the adaptive model to benefit from the stable guidance of the well-trained DGNN while gradually aligning with test-time dynamics via online updates. The memory bank also helps reduce confirmation bias during online adaptation. If the adaptive model relies only on its current prediction, an incorrect high-confidence prediction may be immediately reinforced by the consistency objective, leading to error accumulation over time. By aggregating predictions from recent timestamps, the memory bank provides a temporally smoothed target that is less sensitive to abrupt noise at a single snapshot. At the same time, the exponential weighting scheme prevents outdated predictions from dominating the pseudo-label, allowing the model to remain responsive to newly emerging graph patterns.

Drift-Aware Exponential Moving Average. Exponential moving average (EMA) is a widely used technique for stabilizing model updates by maintaining a smoothed version of model parameters: $\theta \leftarrow \alpha \cdot \theta + (1 - \alpha) \cdot \theta$, where α is the momentum coefficient that controls how much past parameters influence the update. While exponential moving average (EMA) based models are highly effective in static domains, they become vulnerable in dynamic graphs. In particular, EMA ‘misremembers’ outdated patterns or generates unstable pseudo-labels when the graph structure changes drastically — e.g., when new edges emerge or previous neighbors disappear. Instead of using a fixed momentum, we modulate the EMA coefficient by the structural drift between two successive graph snapshots:

$$\theta_t \leftarrow \alpha_t \theta_{t-1} + (1 - \alpha_t) \theta_t, \quad (4)$$

where the time-dependent momentum α_t is defined as

$$\alpha_t = \alpha_{\text{base}} \exp(-\gamma d_t), \quad \alpha_{\text{base}} \in (0, 1), \gamma > 0. \quad (5)$$

The drift term d_t quantifies the extent of graph changes between time steps $t - 1$ and t , and can be defined using either of the following two interchangeable formulations.

(a) *Edge-set Jaccard distance.*

$$d_t = \frac{|\mathcal{E}_t \triangle \mathcal{E}_{t-1}|}{|\mathcal{E}_t \cup \mathcal{E}_{t-1}|}, \quad (6)$$

where $\triangle$ denotes symmetric difference.

(b) *Adjacency-matrix Frobenius distance.*

$$d_t = \frac{1}{|V|^2} \|\mathbf{A}_t - \mathbf{A}_{t-1}\|_F^2, \quad (7)$$

with $\mathbf{A}_t$ the binary adjacency matrix at time t .

A larger d_t (i.e., more drastic structural change) yields a smaller α_t , which causes the adaptive model to rely less on the fixed well-trained DGNN and update more aggressively—mitigating the risk of ‘misremembering’ obsolete

structural patterns. Conversely, when the graph is relatively stable, α_t stays close to the base momentum α_{base} , allowing the adaptive model to retain long-term knowledge from the well-trained DGNN.

Memory-Bank Pseudo Labeling. To reduce prediction variance and enhance stability during online adaptation, we maintain a fixed-length *memory bank* that stores the most recent $K+1$ predictions from the evolving model: $\{\mathbf{y}_t, \mathbf{y}_{t-1}, \dots, \mathbf{y}_{t-K}\}$. Here, $\mathbf{y}_{t-k}$ represents the predicted logits (or probabilities) produced by the online adaptive model at time $t-k$, which was initialized from the well-trained DGNN. By aggregating historical predictions, the memory bank provides temporal context that smooths label estimation and mitigates the impact of noisy fluctuations in any single time step.

We construct the memory-enhanced pseudo label as an exponentially weighted average of past predictions:

$$\hat{\mathbf{y}}_t = \sum_{k=0}^K w_{t-k} \mathbf{y}_{t-k}, \quad w_k = \frac{e^{-\lambda k}}{\sum_{j=0}^K e^{-\lambda j}}, \quad (8)$$

where $\lambda > 0$ controls the decay rate, giving more weight to recent predictions while down-weighting older ones.

To further stabilize the pseudo-supervision signal, we blend the memory-aggregated prediction $\hat{\mathbf{y}}$ with the recent output $\mathbf{y}_t$ of the online model using a convex interpolation:

$$\tilde{\mathbf{y}}_t = \omega \cdot \hat{\mathbf{y}} + (1 - \omega) \cdot \mathbf{y}_t, \quad (9)$$

where $\omega \in [0, 1]$ is a fixed mixing coefficient. This formulation ensures that the pseudo label reflects long-term historical trends (via memory). The final pseudo label $\tilde{\mathbf{y}}_t$ is then used as the reference target to guide the online model via consistency-based adaptation objectives.

C. Consistency-Guided Online Adaptation

Most existing test-time adaptation methods assume i.i.d. samples or stationary feature shifts, which fail in dynamic graphs where both topology and features evolve continuously. To address this, our consistency-guided online adaptation explicitly aligns predictions across time, mitigating structural dependency drift and ensuring stable adaptation under gradual temporal changes. To adapt to this evolving environment, DGOTTA applies a consistency-guided online adaptation strategy that enforces alignment between current predictions and temporally smoothed pseudo labels across consecutive time steps.

Prediction Consistency Loss. We enforce alignment between the adaptive model’s prediction and the memory bank-based pseudo label at the current time step t using Kullback–Leibler (KL) divergence:

$$\mathcal{L}_{\text{con}} = - \sum_i \mathbf{y}_t(i) \cdot \log \mathbf{y}'_t(i), \quad (10)$$

where $\mathbf{y}_t$ is the pseudo-label derived from memory-based interpolation (Eq. (8)), $\mathbf{y}'_t$ is the current of the adaptive model.

Temporal Smoothness Loss. To regularize prediction stability over time, we introduce a temporal consistency loss that

penalizes deviations between the adaptive model’s current output and its previous-step output:

$$\mathcal{L}_{\text{temp}} = \|\mathbf{y}'_t - \mathbf{y}'_{t-1}\|^2. \quad (11)$$

This encourages DGOTTA to produce temporally smooth predictions, unless significant distributional drift is observed, and The total loss at test time combines both terms:

$$\mathcal{L} = \mathcal{L}_{\text{con}} + \lambda \cdot \mathcal{L}_{\text{temp}}, \quad (12)$$

where λ is a balancing coefficient. This objective allows DGOTTA to adapt online while preserving short-term semantic consistency and long-term temporal stability both critical under structural drift.

D. Overall Optimization Procedure

Algorithm 1 summarizes the overall online adaptation procedure of DGOTTA. At each test timestamp, the incoming graph snapshot is first augmented with temporal-aware feature and edge perturbations. The adaptive model then generates predictions, which are combined with memory-based pseudo labels to compute the consistency and temporal smoothness objectives. After updating the model parameters, DGOTTA further applies a drift-aware EMA update based on the structural discrepancy between consecutive graph snapshots, and stores the latest prediction in the memory bank for future adaptation.

Algorithm 1 DGOTTA: Temporal Memory-Aware Online Test-Time Adaptation

- 1: **Input:** Pretrained DGNN f_{θ^*} ; Graph stream $\{G_t = (V_t, E_t, \mathbf{x}_t)\}_{t=1}^T$; Hyperparameters: $K, \omega, \lambda_{\text{temp}}, \alpha_{\text{base}}, \gamma, W, p_{\text{drop}}$
 - 2: **Output:** Adapted model f_{θ}
 - 3: Initialize $\theta \leftarrow \theta^*$; MemoryBank $\leftarrow \emptyset$
 - 4: **for** $t = 1$ to T **do**
 - 5: $\tilde{\mathbf{x}}_t \leftarrow \text{TemporalFeatureAugment}(\mathbf{x}_t)$ // Eq.(2)
 - 6: $\tilde{E}_t \leftarrow \text{TemporalEdgePerturb}(E_t, W, p_{\text{drop}})$ // Eq.(3)
 - 7: $\mathbf{y}_{\text{pred}} \leftarrow f_{\theta}(\tilde{X}_t, \tilde{E}_t)$
 - 8: $\mathbf{y}_{\text{memory}} \leftarrow \text{MemoryWeightedAvg}(\text{MemoryBank}, \lambda)$ // Eq.(8)
 - 9: $\tilde{\mathbf{y}} \leftarrow \omega \cdot \mathbf{y}_{\text{memory}} + (1 - \omega) \cdot \mathbf{y}_{\text{pred}}$ // Eq.(9)
 - 10: Compute $L_{\text{con}}, L_{\text{temp}}, L$ // Eqs. (10, 11, 12)
 - 11: Update model: $\theta \leftarrow \theta - \eta \nabla_{\theta} L$
 - 12: Compute graph drift d_t from E_t, E_{t-1} // Eq.(6) or (7)
 - 13: $\alpha_t \leftarrow \alpha_{\text{base}} \cdot \exp(-\gamma \cdot d_t)$ // Eq.(5)
 - 14: $\theta \leftarrow \alpha_t \cdot \theta + (1 - \alpha_t) \cdot \theta^*$ // Eq.(4)
 - 15: Update MemoryBank with $\mathbf{y}_{\text{pred}}$
 - 16: **end for**
-

V. EXPERIMENT

A. Experiment Setting

Dataset. We conduct experiments on three real-world dynamic graph datasets: Reddit, Wikipedia (Wiki), and MOOC, which are widely used in temporal graph learning benchmarks [48].

- **Wikipedia:** consists of edits on Wikipedia pages over one month. Editors and Wiki pages are modelled as nodes, and the timestamped posting requests are edges.
- **Reddit:** models subreddits’ posted spanning one month, where the nodes are users or posts and the edges are the timestamped posting requests.
- **MOOC:** is a student interaction network formed from online course content units such as problem sets and videos. Each edge is a student accessing a content unit and has 4 features.

The details of dataset statistics are in Table I.

For each dataset, we adopt a 10%/10%/80% chronological split into training, validation, and test sets. This temporal partitioning ensures that the model is trained on early graph snapshots and evaluated on future unseen dynamics, which aligns with the real-world scenario of online deployment. We choose this split to simulate the practical setting of test-time adaptation, where the model is trained on a small historical portion and must generalize to a long sequence of incoming test data with potential distribution shift. The large test portion (80%) allows us to observe long-horizon adaptation performance and evaluate robustness under temporal drift.

Baselines. Since our method is the first online test-time adaptation approach on dynamic graph, there is no available baseline for direct comparison. We compare our method against two categories of test-time adaptation (TTA) baselines: (1) a generic TTA approach, TENT [17], and (2) graph-specific TTA methods, including GTrans [31], SOGA [32], and MATCHA [49]. We use this baselines with another feature time to do dynamic graph test-time adaptation at each timestamp to compare fairly. Moreover, we include a self-supervised dynamic graph learner, DDGCL [50], and a topological-aware TTA method, IDOL [51], for comprehensive comparison. To ensure a fair evaluation, we restrict our comparison to TTA settings, where each method adapts a pretrained model to a target dynamic graph during adaptation. We consider four DGNN as four representative backbones, i.e., GraphMixer [43] (GMixer. for short), DyGFormer [44] (DyGF. for short), TGN [42], and TGAT [41] for the node classification task. All models are pretrained on the training dynamic graph using Empirical Risk Minimization (ERM), and test-time adaptation is applied to the evolving target graph. We evaluate model performance based on the Area Under the ROC Curve (AUC) for node classification tasks.

Implementation and fairness protocol. For a fair comparison, all methods are initialized from the same ERM-pretrained DGNN backbone and evaluated on the same chronological test stream. No method is allowed to access test labels or future graph snapshots during adaptation. For static graph TTA baselines, we adapt them to the dynamic setting by applying their unsupervised adaptation objectives independently at each incoming timestamp. For graph transformation-based methods, the transformation is performed on the current test snapshot before prediction, while model-update-based methods update parameters online after observing the current unlabeled snapshot. All hyperparameters are selected based on the validation

TABLE I: Statistical details of the experimental datasets.

Dataset	Domain	# Nodes	Total Edges	Unique Edges	Unique Steps	Time Granularity	Duration
Wikipedia	Social	9,227	157,474	18,257	152,757	Unix timestamp	1 month
Reddit	Social	10,984	672,447	78,516	669,065	Unix timestamp	1 month
MOOC	Interaction	7,144	411,749	178,443	345,600	Unix timestamp	17 months

split, and the same backbone architecture and evaluation metric are used across all methods. For DGOTTA, we use memory size $K = 200$, interpolation factor $\omega = 0.1$, temporal consistency weight $\lambda_{\text{temp}} = 0.1$, EMA base momentum $\alpha_{\text{base}} = 0.1$, and drift-aware decay factor $\gamma = 0.1$ unless otherwise specified. These values are selected according to validation performance and further analyzed in the parameter sensitivity study.

B. Main Results

Table II reports the AUC scores of different test-time adaptation methods across three dynamic graph benchmarks and four DGNN architectures. Overall, our proposed method DGOTTA consistently achieves the best performance across all datasets and backbones, highlighting its effectiveness in handling both temporal and structural distribution shifts.

On the Wikipedia dataset, DGOTTA achieves the highest AUC on all backbones, with notable improvements of +8.1% on GraphMixer, +6.1% on TGN, and +2.2% on TGAT compared to the best-performing baseline. On Reddit, while some baselines such as GTrans fail due to out-of-memory (OOM) issues on certain backbones, DGOTTA remains robust and achieves state-of-the-art results (e.g., 0.6036 on GraphMixer and 0.6011 on TGAT). Similarly, on MOOC, DGOTTA achieves consistent gains across all architectures, demonstrating strong generalization even in the presence of sparse and noisy temporal interactions.

Traditional methods like ERM and TENT show limited adaptability in dynamic graphs, reflecting their inability to update representations under evolving graph distributions. Graph-specific TTA methods such as GTrans and SOGA improve over ERM in some cases but suffer from poor stability across datasets, suggesting that they fail to fully capture temporal dynamics. MATCHA, while benefiting from consistency-based self-training, is less effective under severe temporal drifts, especially on Reddit, where its performance drops compared to Wikipedia.

These results demonstrate that DGOTTA is not only effective but also model-agnostic: it provides consistent performance gains across diverse DGNN backbones without retraining or hyperparameter tuning on specific datasets. This robustness, combined with its scalability, underlines the practical value of DGOTTA for real-world dynamic graph applications where temporal distribution shifts are inevitable.

Backbone-wise observations. We further observe that the effectiveness of DGOTTA is consistent across different types of DGNN architectures. For attention-based backbones such as TGAT and DyGFormer, DGOTTA improves adaptation by providing temporally smoothed pseudo-labels and reduc-

ing prediction inconsistency across consecutive snapshots. For memory-based or mixer-style models such as TGN and GraphMixer, DGOTTA further complements their temporal representation learning by explicitly modeling test-time structural drift and pseudo-label stability. This suggests that DGOTTA does not rely on a specific architectural design of the backbone. Instead, it acts as a general online adaptation layer that can be integrated with different DGNNs to improve robustness under evolving test distributions.

Another important observation is that several existing baselines perform well on certain datasets but are unstable across backbones. For example, some graph-specific TTA methods achieve competitive results on Wikipedia but degrade on Reddit or MOOC, indicating that adaptation strategies designed for static graph shifts may not generalize well to dynamic graph streams. In contrast, DGOTTA consistently achieves the best or near-best performance across all backbone-dataset combinations, demonstrating stronger robustness to both temporal and structural shifts.

C. Ablation Study

We conduct an ablation study to assess the contribution of individual components in DGOTTA, with results summarized in Table III. Starting from DGOTTA-base, which applies a pretrained DGNN directly to the test graph stream without adaptation, we observe clear performance degradation under temporal and structural distribution shifts.

Adding the temporal consistency (TC) loss provides modest but consistent improvements, indicating that enforcing prediction smoothness helps reduce noisy updates in dynamic settings. Incorporating memory-bank pseudo labeling (MBPL) leads to substantial gains by stabilizing pseudo labels through historical prediction aggregation. Temporal-aware augmentation (TAA) further improves performance, demonstrating the importance of modeling temporal variations in node features and interaction structures. The drift-aware EMA (DAEMA) module brings additional benefits by adapting the update rate to structural changes, thereby mitigating catastrophic forgetting.

Combining all components yields the best performance across datasets and backbones, confirming that the proposed modules are complementary and jointly enable robust online test-time adaptation on dynamic graphs.

D. Analysis of Distribution Shift Mitigation.

We further analyze the ablation results from the perspective of distribution shift mitigation in dynamic graphs. Each component in DGOTTA is designed to address a specific challenge induced by temporal distribution shifts.

TABLE II: Test AUC on three benchmark datasets under different backbones. ‘OOM’ indicates out-of-memory failure.

Methods	Wikipedia				Reddit				MOOC			
	GMixer	DyGF	TGN	TGAT	GMixer	DyGF	TGN	TGAT	GMixer	DyGF	TGN	TGAT
ERM	0.5018	0.7311	0.5855	0.5397	0.5037	0.5254	0.5272	0.5566	0.5064	0.5224	0.5666	0.5824
DDGCL [50]	0.4646	0.4168	0.5181	0.4677	0.4833	0.5043	0.5048	0.4913	0.5708	0.5250	0.5354	0.5166
IDOL [51]	0.5335	0.5016	0.4379	0.5306	0.5112	0.5083	0.5420	0.5133	0.5738	0.4930	0.4932	0.4973
TENT [17]	0.5140	0.5055	0.5175	0.4999	0.5014	0.4999	0.5002	0.5011	0.4999	0.5002	0.5032	0.4999
GTrans [31]	0.5175	0.7731	0.5005	0.4986	OOM	0.4560	OOM	OOM	OOM	0.4884	0.5140	OOM
SOGA [32]	0.5233	0.7736	0.6137	0.4788	0.5114	0.4684	0.5090	0.4952	0.5712	0.5177	0.5032	0.5222
MATCHA [49]	0.5259	0.7788	0.6137	0.5509	0.4785	0.5139	0.5481	0.5984	0.4931	0.4995	0.5130	0.4986
DGOTTA(ours)	0.5827	0.7923	0.6433	0.5614	0.6036	0.5613	0.5836	0.6011	0.6233	0.5428	0.6133	0.6212

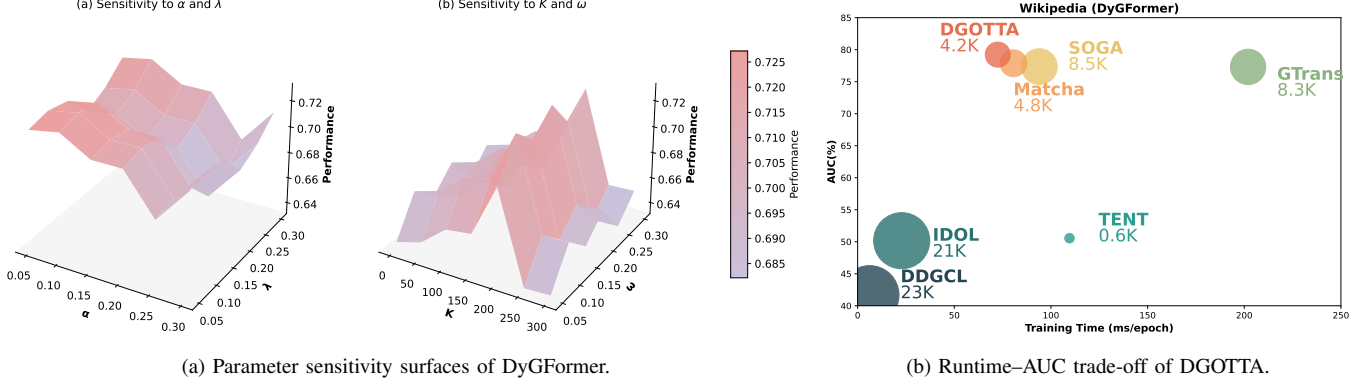


Fig. 3: Parameter sensitivity and efficiency analysis.

TABLE III: Ablation study on dynamic graph benchmarks under different backbones. DGOTTA-base is our proposed method without temporal consistency loss (TC Loss), memory-bank pseudo label (MBPL), temporal-aware augmentation (TAA), and drifter-aware EMA (DAEMA).

Variants	Wikipedia				Reddit				MOOC			
	GMixer	DyGF	TGN	TGAT	GMixer	DyGF	TGN	TGAT	GMixer	DyGF	TGN	TGAT
DGOTTA-base	0.5009	0.6008	0.5009	0.5197	0.5012	0.5302	0.5212	0.5499	0.5005	0.5008	0.5003	0.5233
+TC Loss	0.5043	0.5976	0.5239	0.5105	0.5025	0.5289	0.5267	0.5149	0.5018	0.5185	0.5021	0.5338
+MBPL	0.4943	0.7658	0.5232	0.5097	0.5031	0.5287	0.5267	0.5350	0.5030	0.5069	0.5099	0.5384
+TAA	0.5442	0.7471	0.5950	0.5362	0.5520	0.5308	0.5565	0.5535	0.5472	0.5077	0.5131	0.5158
+DAEMA	0.5708	0.6998	0.6186	0.5507	0.5730	0.5339	0.5729	0.5518	0.5651	0.5081	0.5083	0.5445
DGOTTA(ours)	0.5827	0.7923	0.6433	0.5614	0.6036	0.5613	0.5836	0.6011	0.6233	0.5428	0.6133	0.6212

Specifically, temporal-aware augmentation (TAA) targets semantic and temporal feature shifts by perturbing node representations according to interaction recency, thereby reducing the mismatch between training-time and test-time feature distributions. The consistent performance gains observed after adding TAA indicate that explicitly modeling temporal feature drift is crucial under evolving graph streams.

The memory-based pseudo-labeling module is designed to alleviate prediction instability caused by non-stationary graph dynamics. As shown in the ablation results, incorporating memory aggregation leads to more stable improvements, particularly on datasets with pronounced temporal variability, suggesting its effectiveness in smoothing noisy predictions under distribution shifts.

Finally, the drift-aware EMA mechanism explicitly addresses catastrophic forgetting in online adaptation. Unlike

conventional EMA schemes that assume stationary update dynamics, the proposed drift-aware EMA adjusts the update rate according to structural changes between consecutive snapshots. The additional gains brought by this component demonstrate its importance in preserving useful historical knowledge while adapting to evolving graph structures.

Another observation is that the components are not isolated from each other. Temporal-aware augmentation increases the diversity of test-time views, but this may also introduce noisy predictions if used without stable pseudo-supervision. The memory bank alleviates this issue by aggregating recent predictions into smoother targets. Similarly, drift-aware EMA controls how strongly the model should retain historical knowledge under different levels of structural change. Therefore, the full model benefits from the interaction among augmentation, memory smoothing, and drift-aware updating,

which explains why combining all modules achieves the strongest performance.

Overall, the ablation results confirm that the performance improvements of DGOTTA stem from its ability to mitigate multiple types of distribution shifts in dynamic graphs, rather than from generic regularization effects.

E. Parameter Analysis

We conduct a parameter sensitivity analysis to investigate several key hyperparameters, including the memory bank size K , reference-adaptation interpolation factor ω , temporal consistency weight λ , and EMA update ratio α . Figure 3a shows the AUC performance on the Wikipedia dataset under different parameter settings.

Memory Size K . As shown in the first plot, increasing the memory bank size K leads to performance gains for DyGFormer up to a peak at $K = 200$, while GraphMixer achieves the best performance around $K = 200$ but drops sharply after that. This indicates that a moderate memory size helps stabilize pseudo-labels, while excessive memory may introduce outdated or noisy predictions.

Interpolation Factor ω . A smaller ω (i.e., giving more weight to memory bank predictions) leads to improved results, with optimal performance around $\omega = 0.1$. This confirms the effectiveness of aggregating historical knowledge.

Temporal Consistency Weight λ . From the third plot, we find that setting $\lambda = 0.1$ yields the best performance. This confirms that enforcing temporal smoothness helps the adaptive model generalize better across evolving graph structures, but excessive regularization may hinder flexibility.

EMA Update Ratio α . In the fourth plot, performance peaks around $\alpha = 0.1$, which balances stability and adaptability. Too small α slows adaptation, while too large α makes well-trained model overfit noisy test samples.

Overall, the consistent trend across different architectures verifies the robustness of our hyperparameter design.

F. Efficiency Analysis

To evaluate the efficiency of different test-time adaptation methods, we report the average inference time per timestamp together with the corresponding AUC scores on the Wikipedia dataset using DyGFormer as the backbone. The results are shown in Figure 3b, where each method is represented by a point reflecting its accuracy–efficiency trade-off.

Conventional methods such as DDGCL and IDOL achieve relatively low inference latency but exhibit limited adaptation performance, indicating a weak capability to handle dynamic structural changes. In contrast, graph-based TTA methods, including GTrans and SOGA, improve performance at the cost of increased computational overhead.

Our proposed DGOTTA achieves the best trade-off between efficiency and performance. Specifically, DGOTTA attains the highest AUC while maintaining a competitive inference time per timestamp, outperforming existing methods in both accuracy and runtime. These results demonstrate that DGOTTA provides an effective and scalable solution for online adaptation in dynamic graph scenarios.

VI. CONCLUSION

In this paper, we proposed DGOTTA, a novel framework for online test-time adaptation on dynamic graphs. DGOTTA is designed to handle both temporal and structural distribution shifts that commonly occur in real-world dynamic environments. By combining time-aware augmentation, a memory-aware model prediction, and consistency-guided online adaptation, DGOTTA enables robust and efficient adaptation. Extensive experiments on real-world datasets and dynamic GNN backbones demonstrate DGOTTA consistently outperforms existing test-time adaptation and self-supervised baselines, achieving state-of-the-art performance with minimal overhead. We believe DGOTTA provides a promising direction for future research on adaptive learning in dynamic graph scenarios, particularly under realistic online inference settings.

ACKNOWLEDGMENTS

This research is supported by the U.S. National Science Foundation and Australia CSIRO joint project: NSF-CSIRO: Towards Interpretable and Responsible Graph Modeling for Dynamic Systems (IIS-2302786). This work has also been supported by the Australian Research Council (ARC) under grants FT210100097 and DP240101547.

REFERENCES

- [1] T. Y. Berger-Wolf and J. Saia, “A framework for analysis of dynamic social networks,” in *Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2006, pp. 523–528.
- [2] D. Greene, D. Doyle, and P. Cunningham, “Tracking the evolution of communities in dynamic social networks,” in *2010 international conference on advances in social networks analysis and mining*. IEEE, 2010, pp. 176–183.
- [3] L. Sun, Z. Zhang, F. Wang, P. Ji, J. Wen, S. Su, and S. Y. Philip, “Aligning dynamic social networks: An optimization over dynamic graph autoencoder,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 6, pp. 5597–5611, 2022.
- [4] W. Song, Z. Xiao, Y. Wang, L. Charlin, M. Zhang, and J. Tang, “Session-based social recommendation via dynamic graph attention networks,” in *Proceedings of the Twelfth ACM international conference on web search and data mining*, 2019, pp. 555–563.
- [5] D. C. Nascimento, B. A. Pimentel, R. M. Souza, L. Costa, S. Gonçalves, and F. Louzada, “Dynamic graph in a symbolic data framework: An account of the causal relation using covid-19 reports and some reflections on the financial world,” *Chaos, Solitons & Fractals*, vol. 153, p. 111440, 2021.
- [6] S. Zhang, T. Suzumura, and L. Zhang, “Dyngraphtrans: Dynamic graph embedding via modified universal transformer networks for financial transaction data,” in *2021 IEEE International Conference on Smart Data Services (SMDS)*. IEEE, 2021, pp. 184–191.
- [7] F. Li, J. Feng, H. Yan, G. Jin, F. Yang, F. Sun, D. Jin, and Y. Li, “Dynamic graph convolutional recurrent network for traffic prediction: Benchmark and solution,” *ACM Transactions on Knowledge Discovery from Data*, vol. 17, no. 1, pp. 1–21, 2023.
- [8] Z. Zhou, Y. Wang, X. Xie, L. Chen, and C. Zhu, “Foresee urban sparse traffic accidents: A spatiotemporal multi-granularity perspective,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 8, pp. 3786–3799, 2020.
- [9] Z. Zhou, Y. Wang, X. Xie, L. Chen, and H. Liu, “Riskoracle: A minute-level citywide traffic accident forecasting framework,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 01, 2020, pp. 1258–1265.
- [10] S. Lan, Y. Ma, W. Huang, W. Wang, H. Yang, and P. Li, “Dstagnn: Dynamic spatial-temporal aware graph neural network for traffic flow forecasting,” in *International conference on machine learning*. PMLR, 2022, pp. 11 906–11 917.

- [11] H. Yuan, Q. Sun, Z. Wang, X. Fu, C. Ji, Y. Wang, B. Jin, and J. Li, "Dg-mamba: Robust and efficient dynamic graph structure learning with selective state space models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 21, 2025, pp. 22 272–22 280.
- [12] H. Yuan, Q. Sun, X. Fu, Z. Zhang, C. Ji, H. Peng, and J. Li, "Environment-aware dynamic graph learning for out-of-distribution generalization," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [13] W. Wang, X. Lin, F. Feng, X. He, M. Lin, and T.-S. Chua, "Causal representation learning for out-of-distribution recommendation," in *Proceedings of the ACM Web Conference 2022*, 2022, pp. 3562–3571.
- [14] J. Liang, R. He, and T. Tan, "A comprehensive survey on test-time adaptation under distribution shifts," *International Journal of Computer Vision*, vol. 133, no. 1, pp. 31–64, 2025.
- [15] V. Jain and E. Learned-Miller, "Online domain adaptation of a pre-trained cascade of classifiers," in *CVPR 2011*. IEEE, 2011, pp. 577–584.
- [16] Y. Sun, X. Wang, Z. Liu, J. Miller, A. Efros, and M. Hardt, "Test-time training with self-supervision for generalization under distribution shifts," in *International conference on machine learning*. PMLR, 2020, pp. 9229–9248.
- [17] D. Wang, E. Shelhamer, S. Liu, B. Olshausen, and T. Darrell, "Tent: Fully test-time adaptation by entropy minimization," *arXiv preprint arXiv:2006.10726*, 2020.
- [18] M. Zhang, S. Levine, and C. Finn, "Memo: Test time robustness via adaptation and augmentation," *Advances in neural information processing systems*, vol. 35, pp. 38 629–38 642, 2022.
- [19] S. Niu, J. Wu, Y. Zhang, Z. Wen, Y. Chen, P. Zhao, and M. Tan, "Towards stable test-time adaptation in dynamic wild world," *arXiv preprint arXiv:2302.12400*, 2023.
- [20] M. Jang, S.-Y. Chung, and H. W. Chung, "Test-time adaptation via self-training with nearest neighbor information," *arXiv preprint arXiv:2207.10792*, 2022.
- [21] R. A. Marsden, M. Döbler, and B. Yang, "Universal test-time adaptation through weight ensembling, diversity weighting, and prior correction," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2024, pp. 2555–2565.
- [22] L. Zeng, J. Han, L. Du, and W. Ding, "Rethinking precision of pseudo label: Test-time adaptation via complementary learning," *Pattern Recognition Letters*, vol. 177, pp. 96–102, 2024.
- [23] M. Boudiaf, R. Mueller, I. Ben Ayed, and L. Bertinetto, "Parameter-free online test-time adaptation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 8344–8353.
- [24] Q. Wang, O. Fink, L. Van Gool, and D. Dai, "Continual test-time domain adaptation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 7201–7211.
- [25] Y. Gan, Y. Bai, Y. Lou, X. Ma, R. Zhang, N. Shi, and L. Luo, "Decorate the newcomers: Visual domain prompt for continual test time adaptation," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 37, no. 6, 2023, pp. 7595–7603.
- [26] J. Song, J. Lee, I. S. Kweon, and S. Choi, "Ecotta: Memory-efficient continual test-time adaptation via self-distilled regularization," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 11 920–11 929.
- [27] D. Lee, J. Yoon, and S. J. Hwang, "Becotta: Input-dependent online blending of experts for continual test-time adaptation," *arXiv preprint arXiv:2402.08712*, 2024.
- [28] J. Chen, X. Zheng, B. Li, Z. Wang, Y. Guo, and F. Xia, "Test-time adaptation for graph learning: A systematic survey," 2026.
- [29] X. Zheng, B. Li, Y. Zheng, Q. Zhang, H. Wang, Y. Liang, A. W.-C. Liew, and S. Pan, "Test-time graph rebirth for gnn generalization under distribution shifts," in *2025 IEEE International Conference on Data Mining (ICDM)*. IEEE, 2025, pp. 973–982.
- [30] X. Zheng, W. Huang, C. Zhou, M. Li, and S. Pan, "Test-time graph neural dataset search with generative projection," in *Forty-second International Conference on Machine Learning*, 2025. [Online]. Available: <https://openreview.net/forum?id=824TCt6CkE>
- [31] W. Jin, T. Zhao, J. Ding, Y. Liu, J. Tang, and N. Shah, "Empowering graph representation learning with test-time graph transformation," *arXiv preprint arXiv:2210.03561*, 2022.
- [32] H. Mao, L. Du, Y. Zheng, Q. Fu, Z. Li, X. Chen, S. Han, and D. Zhang, "Source free graph unsupervised domain adaptation," in *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, 2024, pp. 520–528.
- [33] X. Zheng, Y. Wang, Y. Liu, M. Li, M. Zhang, D. Jin, P. S. Yu, and S. Pan, "Graph neural networks for graphs with heterophily: A survey," *IEEE Transactions on Knowledge and Data Engineering*, 2026.
- [34] M. Ju, T. Zhao, W. Yu, N. Shah, and Y. Ye, "Graphpatcher: mitigating degree bias for graph neural networks via test-time augmentation," *Advances in Neural Information Processing Systems*, vol. 36, pp. 55 785–55 801, 2023.
- [35] G. Chen, J. Zhang, X. Xiao, and Y. Li, "Graphtta: Test time adaptation on graph neural networks," *arXiv preprint arXiv:2208.09126*, 2022.
- [36] X. Wang and M. Zhang, "How powerful are spectral graph neural networks," in *International conference on machine learning*. PMLR, 2022, pp. 23 341–23 362.
- [37] L. Wang, X. Zhang, H. Su, and J. Zhu, "A comprehensive survey of continual learning: Theory, method and application," *IEEE TPAMI*, vol. 46, no. 8, pp. 5362–5383, 2024.
- [38] Z. Feng, R. Wang, T. Wang, M. Song, S. Wu, and S. He, "A comprehensive survey of dynamic graph neural networks: Models, frameworks, benchmarks, experiments and challenges," *IEEE Transactions on Knowledge and Data Engineering*, 2025.
- [39] Y. Zheng, L. Yi, and Z. Wei, "A survey of dynamic graph neural networks," *Frontiers of Computer Science*, vol. 19, no. 6, p. 196323, 2025.
- [40] M. Jin, Y.-F. Li, and S. Pan, "Neural temporal walks: Motif-aware representation learning on continuous-time dynamic graphs," in *Advances in Neural Information Processing Systems*, A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, Eds., 2022. [Online]. Available: <https://openreview.net/forum?id=NqbkIPUKZf7>
- [41] D. Xu, C. Ruan, E. Korpeoglu, S. Kumar, and K. Achan, "Inductive representation learning on temporal graphs," *arXiv preprint arXiv:2002.07962*, 2020.
- [42] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein, "Temporal graph networks for deep learning on dynamic graphs," *arXiv preprint arXiv:2006.10637*, 2020.
- [43] W. Cong, S. Zhang, J. Kang, B. Yuan, H. Wu, X. Zhou, H. Tong, and M. Mahdavi, "Do we really need complicated model architectures for temporal networks?" *arXiv preprint arXiv:2302.11636*, 2023.
- [44] L. Yu, L. Sun, B. Du, and W. Lv, "Towards better dynamic graph learning: New architecture and unified library," *Advances in Neural Information Processing Systems*, vol. 36, pp. 67 686–67 700, 2023.
- [45] B. Li, X. Zheng, M. Jin, C. Wang, and S. Pan, "Test-time gnn model evaluation on dynamic graphs," in *2025 IEEE International Conference on Data Mining (ICDM)*. IEEE, 2025, pp. 437–446.
- [46] X. Zheng, M. Zhang, C. Chen, S. Molaei, C. Zhou, and S. Pan, "Gnnevaluator: Evaluating gnn performance on unseen graphs without labels," *Advances in neural information processing systems*, vol. 36, pp. 33 606–33 623, 2023.
- [47] K. Ding, Z. Xu, H. Tong, and H. Liu, "Data augmentation for deep graph learning: A survey," *ACM SIGKDD Explorations Newsletter*, vol. 24, no. 2, pp. 61–77, 2022.
- [48] S. Kumar, X. Zhang, and J. Leskovec, "Predicting dynamic embedding trajectory in temporal interaction networks," in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 2019, pp. 1269–1278.
- [49] W. Bao, Z. Zeng, Z. Liu, H. Tong, and J. He, "Matcha: Mitigating graph structure shifts with test-time adaptation," in *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. [Online]. Available: <https://openreview.net/forum?id=EpgoFFUM2q>
- [50] S. Tian, R. Wu, L. Shi, L. Zhu, and T. Xiong, "Self-supervised representation learning on dynamic graphs," in *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, ser. CIKM '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 1814–1823. [Online]. Available: <https://doi.org/10.1145/3459637.3482389>
- [51] Z. Zhu, K. Wang, H. Liu, J. Li, and S. Luo, "Topology-monitorable contrastive learning on dynamic graphs," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, ser. KDD '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 4700–4711. [Online]. Available: <https://doi.org/10.1145/3637528.3671777>